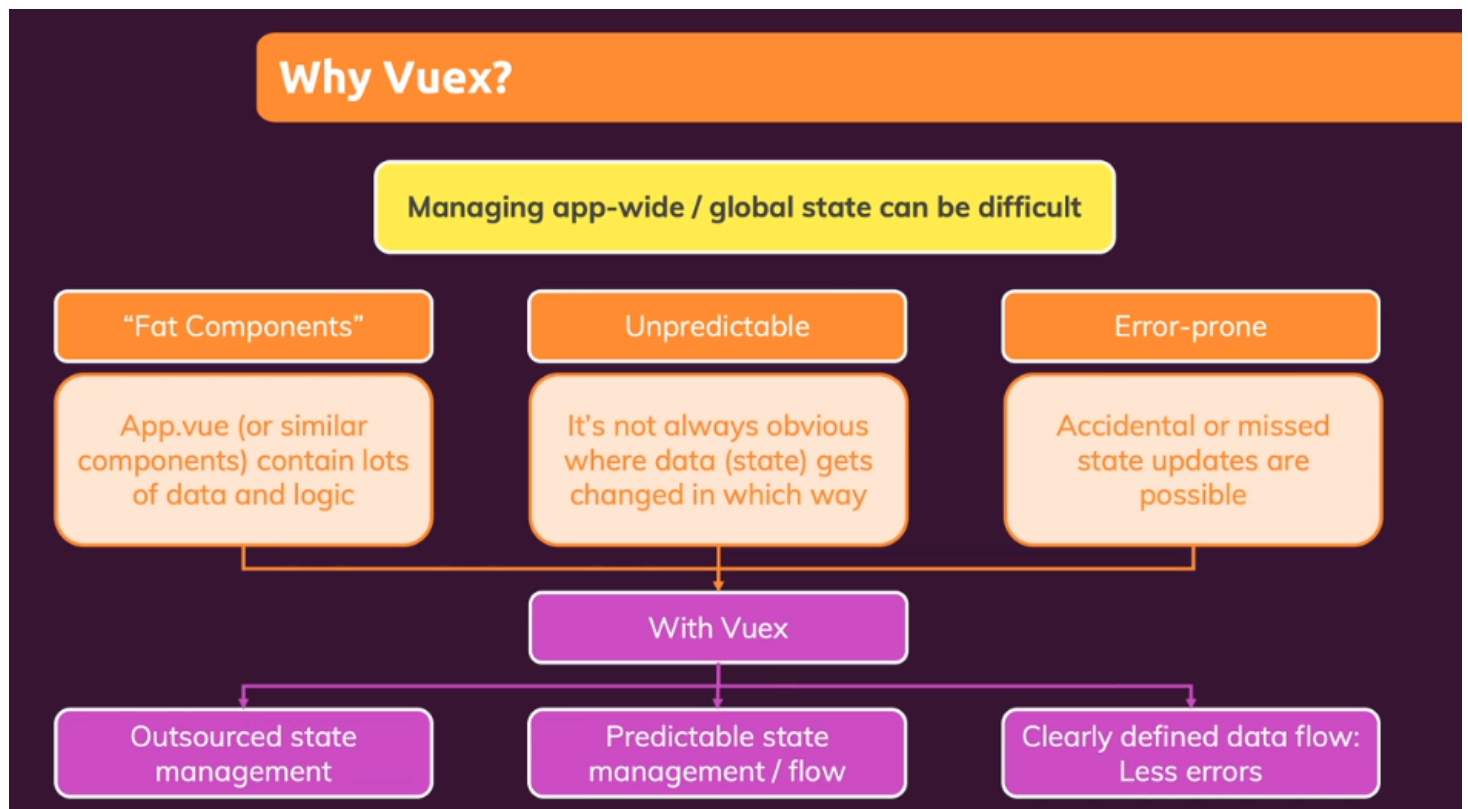


Section 15 - Vuex

Vuex is similar to Redux in React in which we create store to manage state (data).

It is an alternative to provide/inject because using provide and inject can make possible for every component to read data.

It is suggested to skip Vuex and study Pinia



Setting Up Vuex Store

You can setup Vuex Store by following four steps:

Section15.md U

App.vue U

main.js U

BaseConta ...

main.js U

VueLearning > Section15 > vuex-demo-app > src > App.vue > {} script > de

```
1 <template>
2   <BaseContainer title="VUEX">
3     <!-- hi -->
4     <button @click="increaseCount">Increase Counter</button>
5   </BaseContainer>
6 </template>
7
8 <script>
9   import BaseContainer from "../components/BaseContainer.vue";
10
11   Finally, we can use variables stored in state property in any
12   component from this.$store as Vuex injects this store globally
13   using Vue plugin
14   components: {
15     BaseContainer,
16   },
17   computed: {
18     count() {
19       return this.$store.state.counter;
20     },
21   },
22   methods: {
23     increaseCount() {
24       return this.$store.state.counter++;
25     },
26   },
27 };
28 </script>
29
30 <style>
31 * {
32   box-sizing: border-box;
33 }
34
35 html {
36   font-family: sans-serif;
37 }
38
39
```

VueLearning > Section15 > vuex-demo-app > src > main.js > ...

```
1 import { createApp } from "vue";
2 import { createStore } from "vuex";
3
4 import App from "../App.vue";
5
6 const app = createApp(App);
7
8 const store = createStore({
9   state() {
10     return {
11       counter: 0,
12     };
13   },
14 });
15
16 app.use(store);
17
18 app.mount("#app");
19
```

localhost:5173

School

VUEX

10 Increase Counter

1

2

3

4

then use app.use to pass your store object to make Vuex store available globally.

Install Vuex and import createStore from Vuex library

createStore accepts config object that contains state property which acts as data property and in which you define your variable of which you want to maintain its state.

Remember, state is nothing but data with some state that we want to manage.

Elements

Console

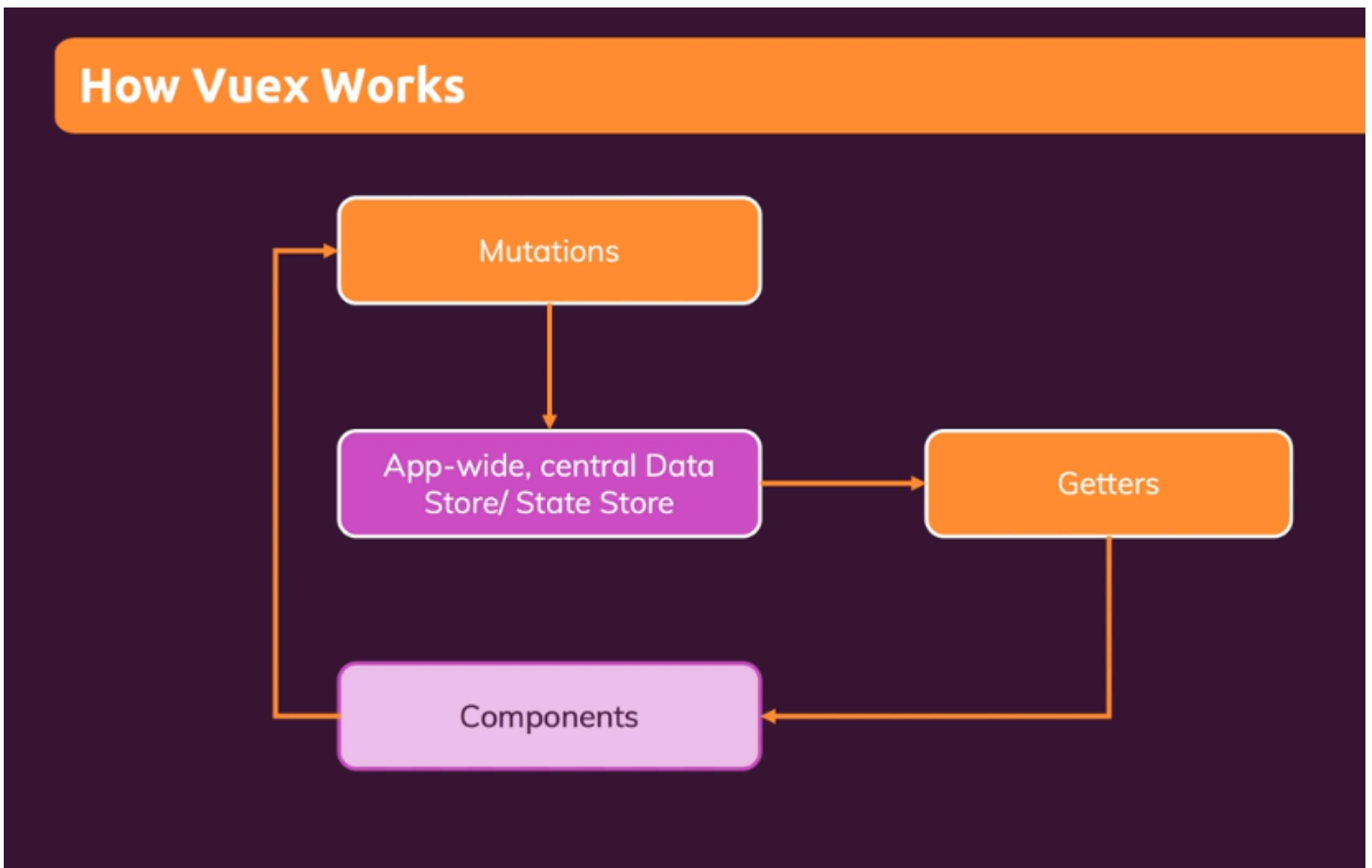
Memory

Sources >>

1 hidden

Relevant data is sent to Google

Mutators and Getters



Mutations

We use mutations to change property value instead of changing directly.

The screenshot displays a development environment with a code editor on the left and a browser on the right. The code editor shows two files: `App.vue` and `main.js`. In `App.vue`, a button labeled "Increase Counter" is defined with a click event that calls `increaseCount`. This method calls `this.$store.commit("increment")`. In `main.js`, a Vuex store is created with an initial state of `counter: 0` and a mutation named `increment` that increments the counter. The browser on the right shows the rendered application with the "VUEX" title and the "Increase Counter" button. A red arrow points from the `commit` method in `App.vue` to the `increment` mutation in `main.js`. Another red arrow points from the `increment` mutation in `main.js` to a red circle containing the number "1". A third red arrow points from the `commit` method in `App.vue` to a red circle containing the number "2".

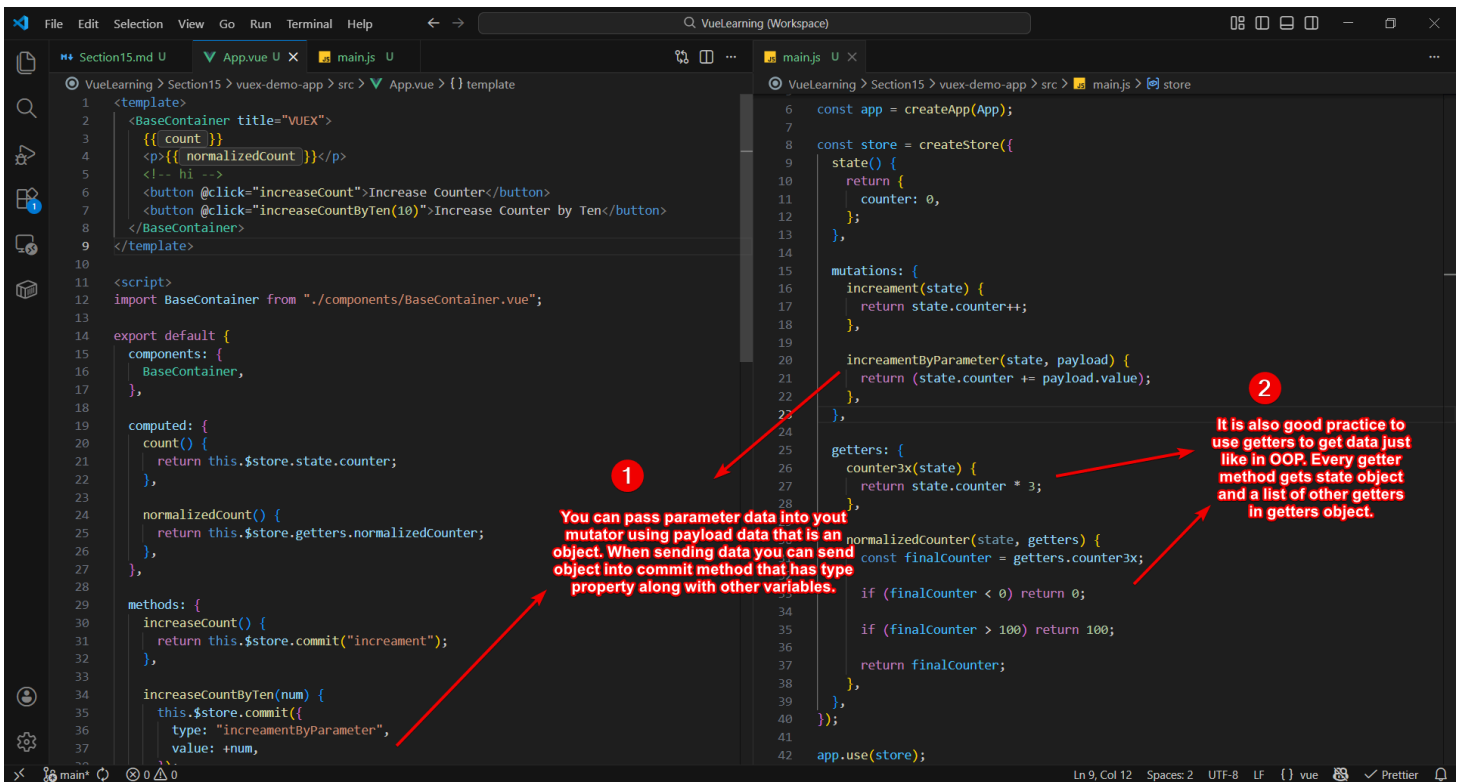
To call the mutation method, we use `this.$store.commit(<Name of Mutation>)` method

Similar, to OOP concept, we don't change properties directly, we create public function to set property value, this concept implies here, we use mutations to define functions that changes property value.

Parameterized Mutators and Getter

You can send data into parameterized mutators using `payload` object.

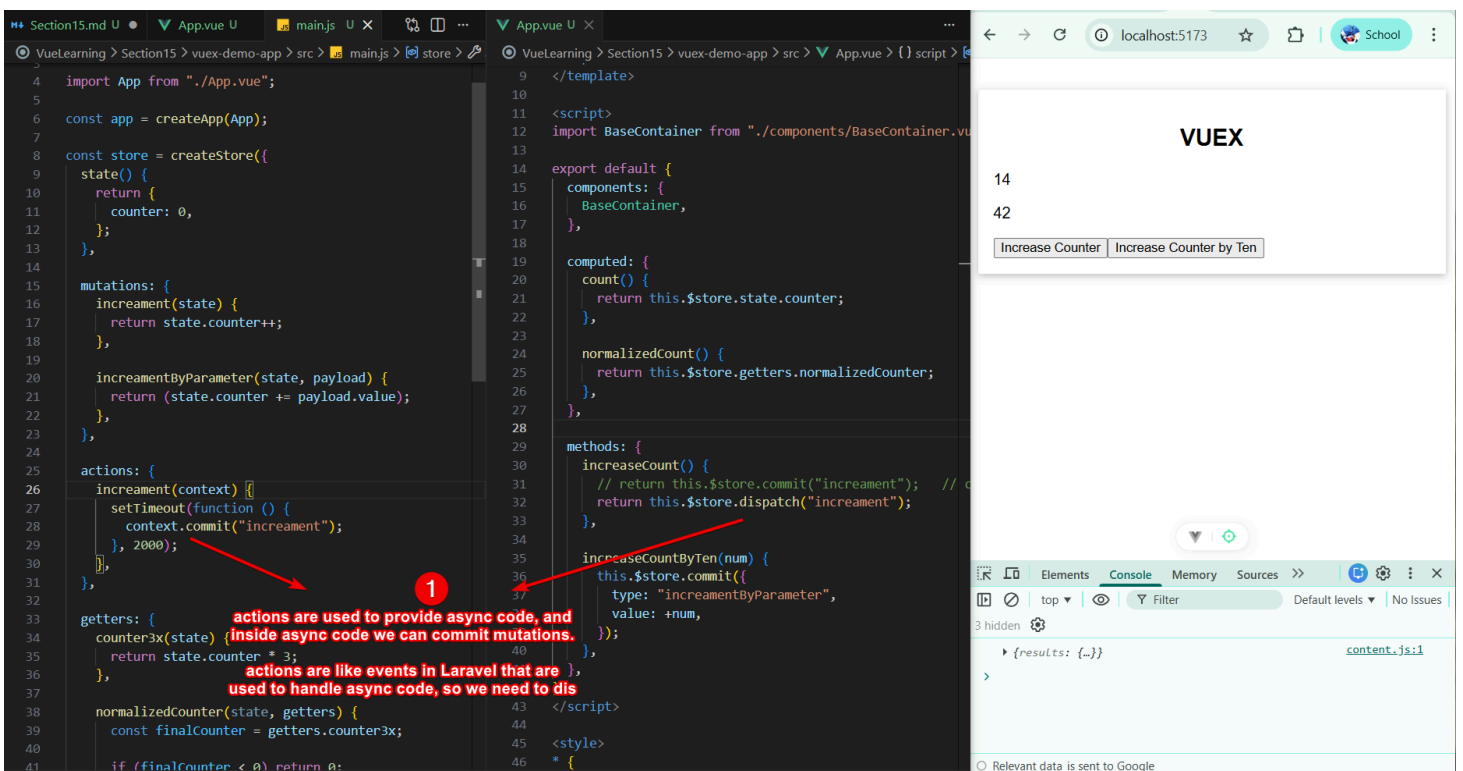
Also, its good practice to get data using getters.



Actions

Actions are similar to Mutators and they are used for two reasons:

1. Instead of mutating the state, actions commit mutations
2. Actions can contain arbitrary **asynchronous operations**



Map Helpers

If you want to access getters into your `computed` property or actions in your `methods` property, you can easily spread map helpers function as they return object containing defined getters and actions.

Vuex provides map helpers function such as mapGetters, mapActions etc that return object that we can spread to get all getters or actions in our computed property or methods.

The mapGetter and mapActions accepts an array of strings of name of getters or actions we want, or an object with key/value pair to customize key names

```
1 <template>
2   <BaseContainer title="VUEX">
3     {{ count }}
4     <p>{{ normalizedCount }}</p>
5     <!-- hi -->
6     <button @click="increaseCount">Increase Counter</button>
7     <button @click="increaseCountByTen(10)">Increase Count by Ten</button>
8   </BaseContainer>
9 </template>
10
11 <script>
12 import BaseContainer from "../components/BaseContainer.vue";
13 import { mapGetters } from "vuex";
14
15 export default {
16   components: {
17     BaseContainer,
18   },
19
20   computed: {
21     // count() {
22     //   return this.$store.state.counter;
23     // },
24     // normalizedCount() {
25     //   return this.$store.getters.normalizedCounter;
26     // },
27     // ...mapGetters(["counter3x", "normalizedCounter"])
28     ...mapGetters({
29       count: "counter3x",
30       normalizedCount: "normalizedCounter",
31     }),
32   },
33
34   methods: {
35     increaseCount() {
36       // return this.$store.commit("increment"); // c
37     }
38   }
39 };
```

```
8 const store = createStore({
9   state: {
10     counter: 0,
11   },
12   mutations: {
13     increment(state) {
14       return state.counter++;
15     },
16     incrementByParameter(state, payload) {
17       return (state.counter += payload.value);
18     },
19   },
20   actions: {
21     increment(context) {
22       context.commit("increment");
23     },
24     incrementByTen(context, payload) {
25       context.commit("incrementByParameter", payload, {
26         meta: {
27           type: "incrementByTen",
28         },
29       });
30     },
31   },
32   getters: {
33     counter3x(state) {
34       return state.counter * 3;
35     },
36     normalizedCounter(state, getters) {
37       const finalCounter = getters.counter3x;
38       if (finalCounter < 0) return 0;
39       if (finalCounter > 100) return 100;
40       return finalCounter;
41     },
42   },
43 });
```

VUEX

63

63

Increase Counter Increase Counter by Ten

2 hidden

content.js:1

{results: [...]}

Modules

When our app grows, our store also grows. To tackle this problem, we need to use modules. Modules are basically an object that has same properties like state, getters, mutators, actions, etc.

Vuex merge all properties together, but name clash can occur and for this reason we have namespace.

The screenshot shows a code editor with the following code in `main.js`:

```
1 import { createApp } from "vue";
2 import { createStore } from "vuex";
3
4 import App from "./App.vue";
5
6 const app = createApp(App);
7
8 const authModule = {
9   namespace: true,
10  state() {},
11  getters: {},
12  mutations: {},
13  actions: {},
14 };
15
16 const store = createStore({
17   modules: {
18     emptyModule: authModule,
19   },
20
21   state() {
22     return {
23       counter: 0,
24     };
25   },
26
27   mutations: {
28     increment(state) {
29       return state.counter++;
30     },
31
32     incrementByParameter(state, payload) {
33       return (state.counter += payload.value);
34     },
35   },
36
37   actions: {
```

Annotations:

- 1: We can define an object that can have all properties that a store config objects has like state, mutations, actions, etc.
- 2: Inside main store config object we can have modules property and inside it define an object with key/value property of having modules object.

Namespace Modules

The screenshot shows a code editor with the following code in `App.vue`:

```
11 <script>
12
13 export default {
14   components: {
15     BaseContainer,
16   },
17
18   computed: {
19
20     // count() {
21     //   return this.$store.state.counter;
22     // },
23
24     // normalizedCount() {
25     //   return this.$store.getters.normalizedCounter;
26     // },
27
28     // ...mapGetters(["counter3x", "normalizedCounter"]),
29     ...mapGetters('emptyModule', {
30       count: "counter2x",
31       normalizedCount: "normalizedCounter",
32     }),
33   },
34 },
35
36 methods: {
37   increaseCount() {
38     // return this.$store.commit("increment"); // commit is for mutators
39     return this.$store.dispatch("emptyModule/increment");
40   },
41
42   increaseCountByTen(num) {
43     this.$store.commit({
44       type: "incrementByParameter",
45       value: +num,
46     });
47   },
48 },
49 </script>
```

Annotations:

- 1: Since Vuex merge getters, mutators etc from module object with these properties in store object, so name clash can occur, and to avoid it we need to set namespace property to true.
- 2: Now the key of module becomes name to access module and getters or actions or mutations
- 3: If these getters and actions were defined in module then we had to access them this way, using `<ModuleName>/<Action/Getter>`

Section 15 - Summary

